

MATERIAL PËRGATITOR PËR PROVIM



PROGRAMIMI PËR PAJISJE MOBILE 2025/2026

Përmbledhje e 12 temave kryesore me diagrame, shembuj praktikë dhe pyetje ushtruese me përgjigje model.

Ligjërues

Arben Lila

Lënda

Programimi për Pajisje Mobile

Institucioni

Kolegji AAB



[Hap materialet Mobile](#)

Material për përsëritje dhe përgatitje për provim.
Bazuar në syllabusin 2025/2026 · Publikuar më 05.10.2026

Si ta përdorni këtë material

Çdo temë ka një ide kryesore, një diagram, koncepte, shembull praktik, pyetje dhe përgjigje model. Përgjigjuni vetë dhe pastaj krahasojeni arsyetimin tuaj.

Qëllimi

Të shpjegoni si funksionon aplikacioni juaj, pse e keni zgjedhur një zgjidhje dhe si e keni provuar. Materiali vlen për stack-un që keni përdorur: Flutter, React Native, Kotlin Compose ose PWA.

Harta e 12 temave



01

Arkitektura mobile dhe Supabase



02

Authentication dhe authorization



03

RLS dhe pronësia e të dhënave



04

CRUD dhe modeli i të dhënave



05

Storage dhe upload media



06

State management dhe UI UX



07

Realtime push dhe Maps



08

REST API dhe Edge Functions



09

Rrjeti gabimet dhe riprovimi



10

AI debugging dhe optimizimi



11

Testimi siguria dhe stabiliteti



12

Deployment dokumentimi dhe demo

1. Lexo konceptin

Lidhe me kodin dhe zgjedhjet e projektit tënd.

2. Shiko diagramin

Shpjego rrjedhën pa lexuar përgjigjen.

3. Përgjigju vetë

Jep konceptin, shembullin dhe arsyetimin.

Pyetjet janë për ushtrim. Data, kohëzgjatja, forma dhe mjetet e lejuara të provimit komunikohen nga profesori për afatin përkatës.

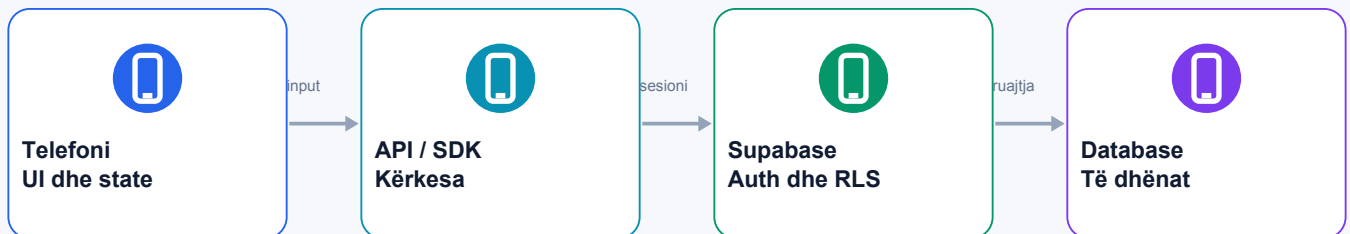


Arkitektura mobile dhe Supabase

Nga veprimi në telefon te të dhënat

Ideja kryesore

Një aplikacion mobile ndan ndërfaqen, logjikën dhe shërbimet cloud; çdo shtresë ka përgjegjësi të qarta.



Konceptet që duhet të dini

- UI-ja merr input dhe shfaq rezultatin; state-i përshkruan gjendjen e ekranit.
- API/SDK lidh klientin me backend-in përmes kërkesave të autorizuara.
- Supabase ofron Auth, PostgreSQL Database, Storage dhe Realtime.
- Kredencialet e privileguara dhe sekretet qëndrojnë në server.

Lidhja me aplikacionin mobile

- Shpjego cilët komponentë ekzekutohen në telefon dhe cilët në cloud.
- Kërkesa kthen rezultat ose gabim; UI-ja përditësohet sipas tij.
- Zgjedhja e Flutter, React Native, Compose ose PWA nuk e ndryshon nevojën për kontroll të qasjes.

Shembull praktik

Studenti shton një shënim: formulari validohet, API-ja kontrollon qasjen dhe databaza ruan shënimin me pronarin.

Pyetje ushtruese

Si kalon një kërkesë nga telefoni te Supabase?

Përgjigjja e modelit

Veprimi në UI thërret logjikën e aplikacionit, e cila përdor API/SDK me sesionin e përdoruesit. Backend-i kontrollon qasjen, përpunon kërkesën dhe kthen rezultat ose gabim. State-i përditësohet dhe UI-ja tregon gjendjen e re. Për shembull, krijimi i një shënimi ruan një rresht me pronarin e tij.

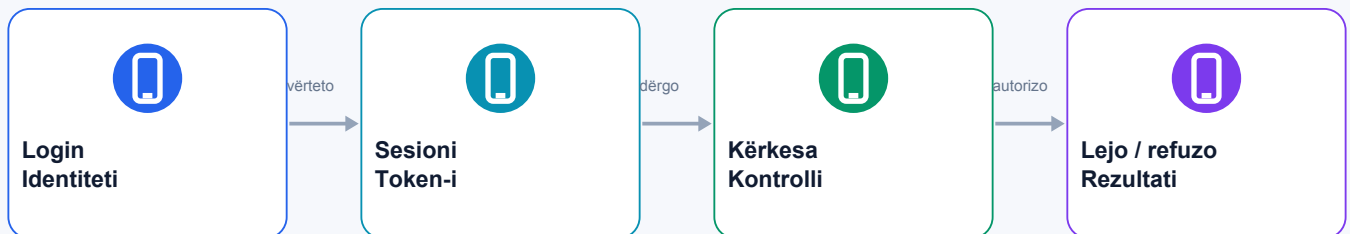


Authentication dhe authorization

Identiteti dhe veprimet e lejuara

Ideja kryesore

Login-i identifikon përdoruesin; kontrolli i qasjes përcakton cilat të dhëna dhe veprime i lejohen.



Konceptet që duhet të dini

- Authentication vërteton identitetin, p.sh. përmes emailit dhe fjalëkalimit.
- Sesioni dhe token-i lidhin kërkesat me përdoruesin e autentifikuar.
- Authorization kontrollon të drejtat për çdo veprim.
- Logout-i mbyll sesionin; gabimet e login-it duhen trajtuar qartë.

Lidhja me aplikacionin mobile

- Një përdorues i kyçur nuk ka automatikisht qasje në të dhënat e të tjerëve.
- Kontrolli i qasjes kryhet në backend/databazë, jo vetëm në ekran.
- Mos ruaj fjalëkalime ose token-e në log-e dhe skedarë publikë.

Shembull praktik

Dy studentë kyçen në të njëjtin aplikacion. Secili mund të lexojë dhe ndryshojë vetëm shënimet e veta.

Pyetje ushtruese

Cili është dallimi mes authentication dhe authorization?

Përgjigjja e modelit

Authentication identifikon përdoruesin, për shembull përmes login-it. Authorization kontrollon çfarë mund të lexojë ose ndryshojë ai përdorues. Login-i nuk i jep automatikisht qasje në të dhënat e të gjithë përdoruesve.

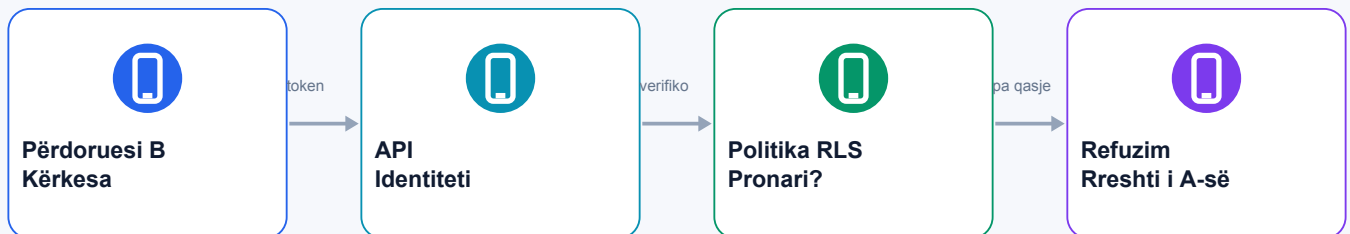


RLS dhe pronësia e të dhënave

Siguri në nivel rreshti

Ideja kryesore

RLS zbaton rregulla në databazë për të kufizuar rreshtat që një përdorues mund të lexojë ose ndryshojë.



Konceptet që duhet të dini

- RLS do të thotë Row Level Security.
- Lejet e tabelës përcaktojnë operacionet; politikat RLS përcaktojnë rreshtat.
- SELECT, INSERT, UPDATE dhe DELETE kërkojnë rregulla sipas nevojës.
- Një fushë owner_id lidh rreshtin me pronarin e autentifikuar.

Lidhja me aplikacionin mobile

- Fshehja e butonit ose filtrimi në UI nuk ndal thirrjet direkte të API-së.
- Politikat duhen provuar edhe me një përdorues tjetër dhe pa login.
- Çelësat secret/service_role kanë privilegje të larta dhe mbahen vetëm në server.

Shembull praktik

A krijon një shënim. B provon ta lexojë dhe ndryshojë me ID-në e tij; databaza duhet ta refuzojë qasjen.

Pyetje ushtruese

Pse një filtër në UI nuk e zëvendëson RLS-në?

Përgjigjja e modelit

Klienti mund të ndryshohet dhe API-ja mund të thirret jashtë UI-së. RLS kontrollon qasjen në databazë, p.sh. vetëm rreshtat ku pronari përputhet me përdoruesin e autentifikuar. Lejet e tabelës dhe politikat për SELECT, INSERT, UPDATE dhe DELETE duhen konfigurim dhe testim. Provo me dy përdorues që njëri nuk lexon ose ndryshon rreshtat e tjetrit.

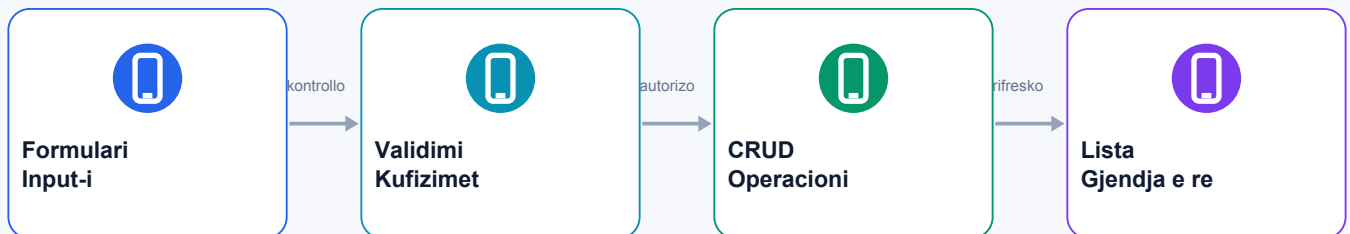


CRUD dhe modeli i të dhënave

Krijim lexim ndryshim fshirje

Ideja kryesore

CRUD përmbledh operacionet bazë të një aplikacioni që menaxhon të dhëna.



Konceptet që duhet të dini

- Create shton të dhëna; Read lexon listën ose detajet.
- Update ndryshon një rresht; Delete e fshin.
- Primary key identifikon rreshtin; foreign key lidh tabela.
- Validimi dhe kufizimet e databazës mbrojnë integritetin e të dhënave.

Lidhja me aplikacionin mobile

- Kontrolllo input-in dhe të drejtat para ndryshimit.
- Përditëso UI-në vetëm sipas rezultatit real të operacionit.
- Një ID e saktë nuk është provë se përdoruesi ka leje për atë rresht.

Shembull praktik

Në notes ruhen id, owner_id, title dhe body. Studenti krijon, lexon, ndryshon dhe fshin shënimin e vet.

Pyetje ushtruese

Përshkruaj CRUD për një aplikacion shënimesh.

Përgjigjja e modelit

Create shton shënimin; Read lexon listën ose detajet; Update ndryshon përmbajtjen; Delete e fshin. Çdo shënim ka ID dhe pronar. Aplikacioni kontrollon input-in, backend-i kontrollon lejet dhe UI-ja shfaq rezultatet ose gabimet.

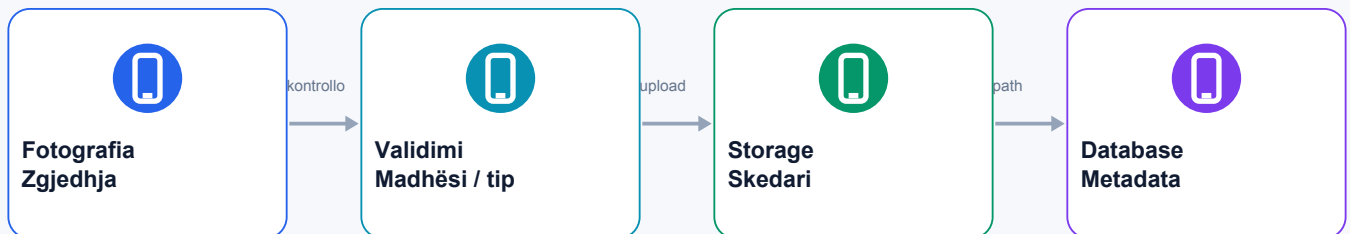


Storage dhe upload media

Skedari dhe metadata

Ideja kryesore

Media ruhet në Storage, ndërsa metadata dhe lidhja me pronarin ruhen në databazë.



Konceptet që duhet të dini

- Bucket-i organizon skedarët dhe rregullat e qasjes.
- Kontrollo madhësinë dhe formatin para dhe gjatë përpunimit.
- Databaza ruan path-in, pronarin dhe të dhënat përshkruese.
- Media private kërkon qasje të autorizuar ose lidhje me afat.

Lidhja me aplikacionin mobile

- Trajto gabimin e upload-it dhe të ruajtjes së metadata-s veçmas.
- Nëse një hap dështon, planifiko pastrim ose riprovim.
- Mos supozo se sukcesi i upload-it garanton sukses të të gjithë operacionit.

Shembull praktik

Aplikacioni ngarkon fotografinë në Storage dhe ruan path-in te profili. Nëse databaza dështon, skedari pastrohet ose riprovohet lidhja.

Pyetje ushtruese

Si e organizon upload-in e një fotografie?

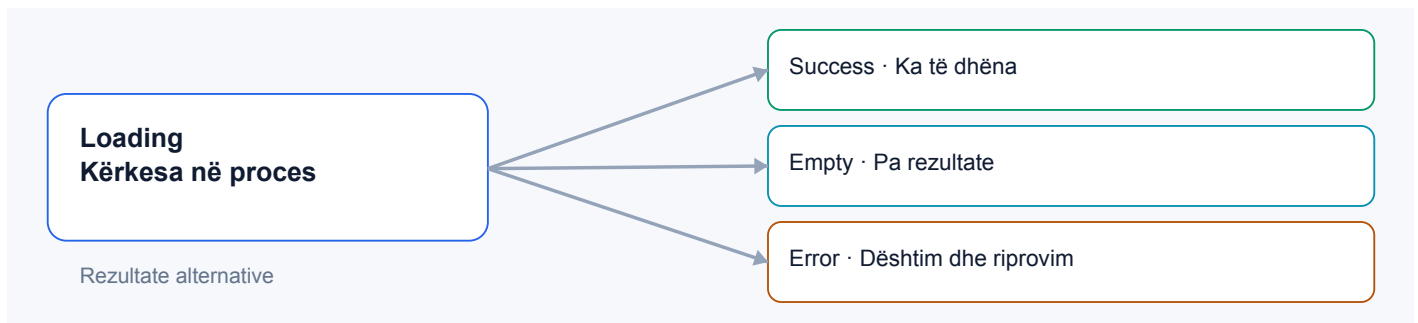
Përgjigjja e modelit

Validoj formatin dhe madhësinë, ngarkoj skedarin në bucket-in e duhur dhe ruaj path-in e tij me pronarin në databazë. Kontrollon lejet e Storage-it dhe trajtoj gabimet. Për fotografi private përdor qasje të kontrolluar; planifiko pastrimin ose riprovimin nëse njëri hap dështon.



Ideja kryesore

Ndërfaqja duhet ta pasqyrojë gjendjen e aplikacionit në mënyrë të kuptueshme, edhe kur kërkesa dështon.



Konceptet që duhet të dini

- State-i përfshin përdoruesin, input-in, të dhënat dhe statusin e kërkesës.
- Loading tregon kërkesën në proces; empty tregon mungesë rezultatesh.
- Success shfaq të dhënat; error shfaq gabimin dhe rrugën për riprovim.
- UI/UX lidhet me ndërfaqen dhe përvojën e përdoruesit.

Lidhja me aplikacionin mobile

- Gabimi nuk duhet të paraqitet si një listë bosh.
- Ruaj input-in e formularit kur kërkesa dështon.
- Përdor etiketa të qarta, kontrast të lexueshëm dhe butona për ekran të vogël.

Shembull praktik

Lista e shënimeve shfaq tregues loading. Nëse nuk ka shënime, ofron krijimin e të parit; nëse rrjeti dështon, ofron riprovim.

Pyetje ushtruese

Cilat gjendje duhet të ketë një ekran që lexon të dhëna?

Përgjigjja e modelit

Loading gjatë kërkesës, empty kur nuk ka rezultate, success kur ka të dhëna dhe error kur kërkesa dështon. Ekranit tregon mesazh të qartë dhe mundësi riprovimi. Gjendja e gabimit nuk duhet të duket sikur lista është thjesht bosh.

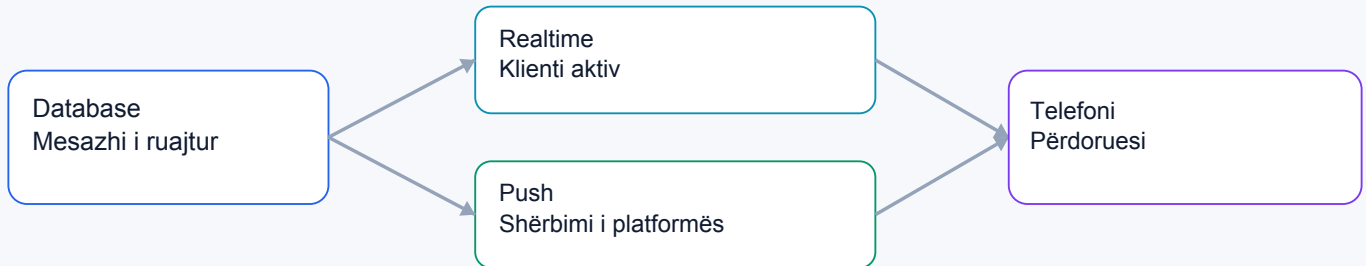


Realtime push dhe Maps

Integrimi me sjellje të ndryshme

Ideja kryesore

Realtime, push notifications dhe Maps zgjidhin nevoja të ndryshme dhe kërkojnë trajtim të lidhjes dhe lejeve.



Konceptet që duhet të dini

- Realtime përditëson klientin përmes një lidhjeje aktive.
- Push notifications përdorin shërbimin e platformës për njoftime.
- Maps shfaq hartën; vendndodhja kërkon leje të përdoruesit.
- Moduli unik duhet të ketë një qëllim të qartë, p.sh. chat, rating ose maps.

Lidhja me aplikacionin mobile

- Trajto refuzimin e lejes për vendndodhje.
- Rilidhja e Realtime nuk garanton se klienti ka marrë çdo ndryshim.
- Push mund të njoftojë kur aplikacioni nuk është aktiv; nuk zëvendëson ruajtjen e të dhënave.

Shembull praktik

Një chat përdor Realtime për mesazhet gjatë përdorimit dhe push për njoftim jashtë aplikacionit. Mesazhet mbeten në databazë.

Pyetje ushtruese

Si ndryshon Realtime nga push notifications?

Përgjigjja e modelit

Realtime përditëson aplikacionin përmes një lidhjeje aktive, për shembull mesazhe të reja në chat. Push notifications përdorin shërbimin e platformës dhe mund të njoftojnë edhe kur aplikacioni nuk është aktiv. Të dyja kanë kërkesa të ndryshme për konfigurim, leje dhe trajtim të lidhjes.

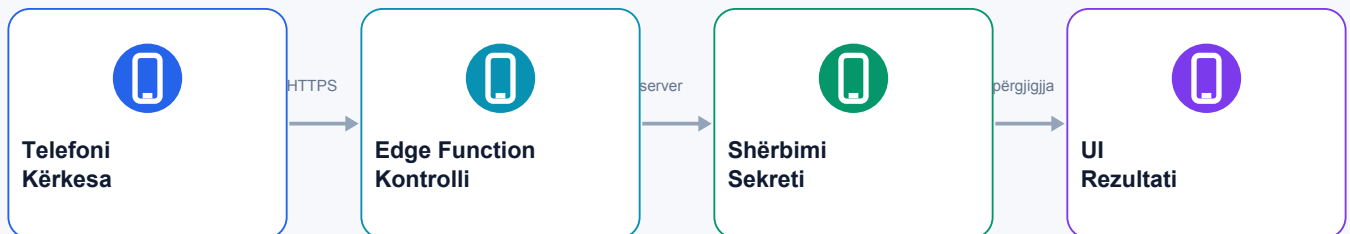


REST API dhe Edge Functions

Logjika dhe sekretet në server

Ideja kryesore

Edge Functions mund të përpunojnë kërkesa në server kur veprimi kërkon sekrete ose kontroll të besueshëm.



Konceptet që duhet të dini

- API përcakton si komunikojnë komponentët; SDK lehtëson përdorimin e saj.
- Në REST, GET lexon, POST krijon, PATCH ndryshon dhe DELETE fshin.
- Edge Function duhet të kontrollojë identitetin, lejet dhe input-in.
- Sekretet e integrameve të jashtme ruhen në server.

Lidhja me aplikacionin mobile

- Mos i vendos çelësat secret/service_role në klient ose repository publik.
- Backend-i nuk duhet t'i besojë identitetit të dërguar thjesht në body.
- Trajto timeout-in dhe përgjigjet e gabimit nga shërbimi i jashtëm.

Shembull praktik

Telefoni kërkon një shërbim të jashtëm përmes Edge Function. Funksioni autorizon kërkesën dhe përdor sekretin e ruajtur në server.

Pyetje ushtruese

Kur e vendos logjikën në një Edge Function?

Përgjigjja e modelit

Kur kërkon një sekret serveri, integrim të jashtëm ose veprim që klienti nuk duhet ta kontrollojë drejtpërdrejt. Funksioni kontrollon identitetin, autorizimin dhe input-in. Çelësat secret/service_role nuk futen në aplikacionin mobil ose në kod publik.

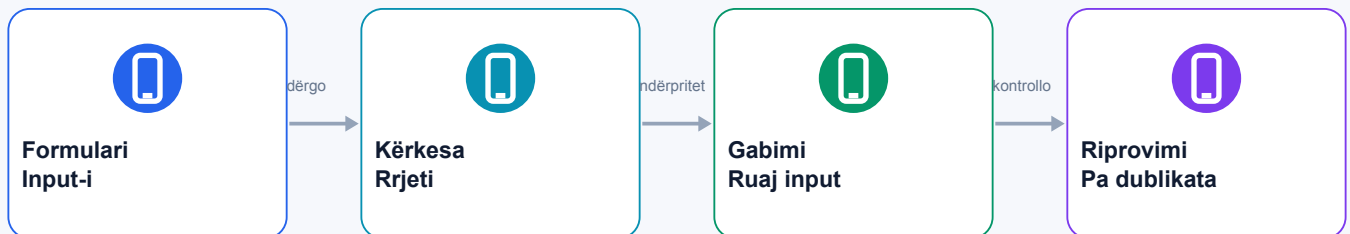


Rrjeti gabimet dhe riprovimi

Sjellje e besueshme në telefon

Ideja kryesore

Aplikacioni duhet të ruajë punën e përdoruesit dhe të tregojë saktë çfarë ndodhi kur lidhja ndërpritet.



Konceptet që duhet të dini

- Timeout-i dhe humbja e rrjetit nuk janë provë e suksesit ose dështimit të ruajtjes.
- Ruaj input-in dhe shfaq mesazh me mundësi riprovimi.
- Cache-i lokal mund të ndihmojë leximin, por mund të jetë i vjetruar.
- Riprovimet e shkrimeve duhet të shmangin dublikatat.

Lidhja me aplikacionin mobile

- Mos shfaq sukses pa konfirmim nga operacioni.
- Në një projekt offline, dallo gjendjen lokale nga ajo e sinkronizuar.
- Shpjego si trajtohen konfliktet dhe kërkesat e përsëritura sipas projektit.

Shembull praktik

Studenti dërgon një shënim dhe rrjeti ndërpritet. Aplikacioni ruan formularin dhe kontrollon nëse shënimi është krijuar para riprovimit.

Pyetje ushtruese

Çfarë bën nëse kërkesa dështon nga humbja e rrjetit?

Përgjigjja e modelit

Ruaj gjendjen e formularit, jap mesazh dhe lejoj riprovim të kontrolluar. Nuk shfaq sukses pa konfirmim. Nëse projekti mbështet cache/offline, dalloj të dhënat lokale nga ato të sinkronizuara dhe shmang krijimin e dublikatave gjatë riprovimit.

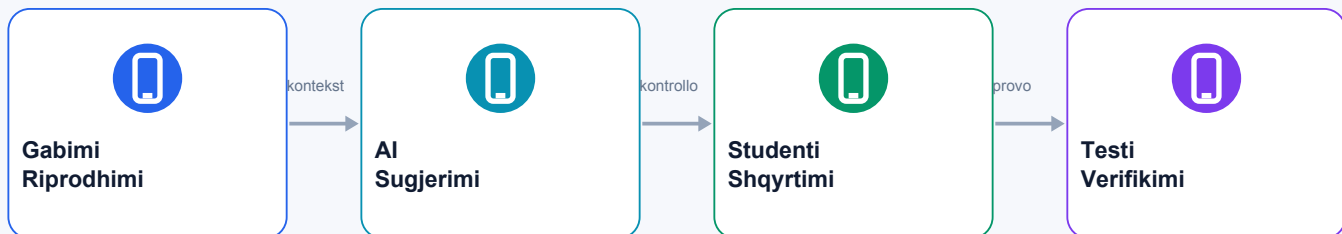


AI debugging dhe optimizimi

Sugjerim verifikim dhe provë

Ideja kryesore

AI-ja mund të ndihmojë zhvillimin, por studenti mbetet përgjegjës për kuptimin dhe verifikimin e kodit.



Konceptet që duhet të dini

- Prompt-i përmban kontekst, qëllim, kufizime dhe sjelljen e pritur.
- Rasti minimal dhe log-et ndihmojnë riprodhimin e gabimit.
- Kontrolllo sugjerimin kundrejt kodit dhe dokumentacionit.
- Mat vonesën, kërkesat dhe madhësinë e të dhënave para optimizimit.

Lidhja me aplikacionin mobile

- Mos dërgo sekrete ose të dhëna reale private te mjetet AI.
- AI-log shënon ndihmën, ndryshimin dhe provën e kryer.
- Një ndryshim që duket i saktë duhet provuar në rrjedhën normale dhe në rastin e gabimit.

Shembull praktik

AI sugjeron korrigjimin e një kërkesë CRUD. Studenti riprodhon gabimin, kontrollon lejet dhe teston korrigjimin me dy përdorues.

Pyetje ushtruese

Si e përdor AI-në për një gabim pa e pranuar verbërisht kodin?

Përgjigjja e modelit

Jap një shembull minimal, sjelljen e pritur dhe gabimin pa sekrete. Shqyrtoj shpjegimin, kontrollon kodin dhe dokumentacionin dhe e testoj korrigjimin. Në AI-log shënoj kërkesën, sugjerimin e përdorur, ndryshimin tim dhe provën që e verifikoi.

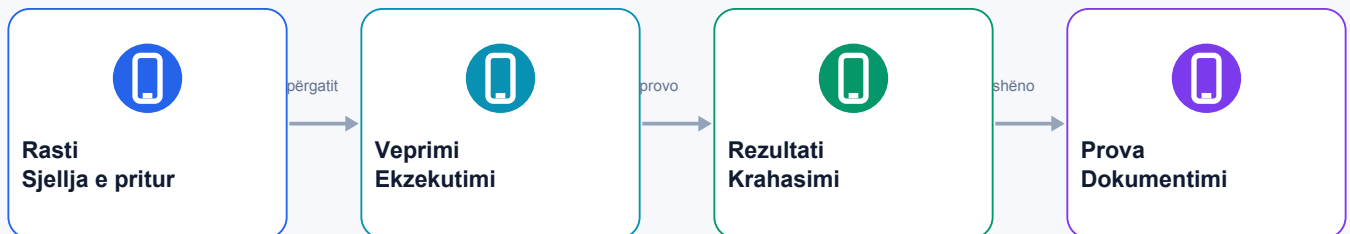


Testimi siguria dhe stabiliteti

Prova që aplikacioni punon

Ideja kryesore

Testimi duhet të mbulojë sjelljen e pritshme dhe veprimet që aplikacioni duhet t'i refuzojë.



Konceptet që duhet të dini

- Testo login/logout, CRUD dhe modulën unik.
- Provo input bosh, input të gabuar dhe rrjet të ndërprerë.
- Përdor dy llogari për të kontrolluar ndarjen e të dhënave.
- Kontrolllo Storage-in, konfigurimin dhe funksionimin në telefon.

Lidhja me aplikacionin mobile

- Një demo që punon vetëm me llogarinë e profesorit nuk mjafton si provë e lejeve.
- Krahajo sjelljen e pritur me rezultatin real dhe dokumento mospërputhjet.
- Pas korrigjimit provo rastin fillestar dhe rrjedhat që preken prej tij.

Shembull praktik

Përdoruesi B kërkon shënimin e A-së: qasja refuzohet. A vazhdon ta lexojë dhe ndryshojë të njëjtin shënim.

Pyetje ushtruese

Si e provon sigurinë dhe stabilitetin e projektit?

Përgjigjja e modelit

Testoj rrjedhën normale dhe rastet e gabimit: login i pasaktë, input bosh, rrjet i ndërprerë dhe përdorues pa leje. Me dy llogari provoj ndarjen e të dhënave. Kontrolllo lejet e Storage-it, konfigurimin pa sekrete dhe funksionimin në telefon.

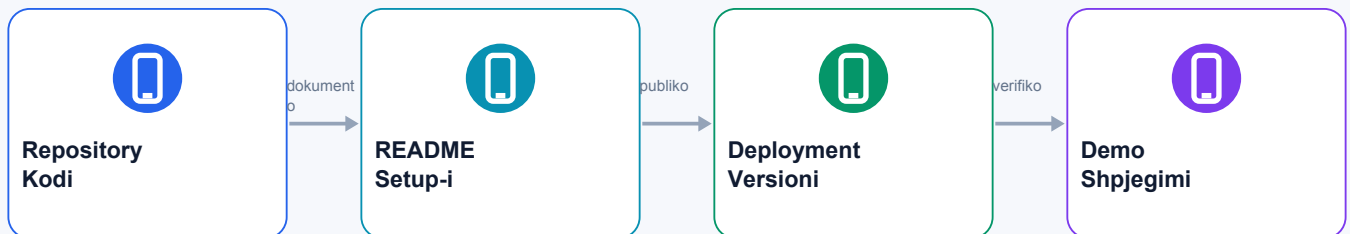


Deployment dokumentimi dhe demo

Një projekt që mund të riprodhohet

Ideja kryesore

Projekti final duhet të funksionojë, të dokumentohet dhe të shpjegohet nga studenti që e dorëzon.



Konceptet që duhet të dini

- Deployment publikon versionin e aplikacionit në mjedisin e zgjedhur.
- README përmban setup, konfigurim dhe udhëzime nisjeje.
- Repository dhe demo ofrojnë evidencë të funksionimit.
- AI-log dhe burimet bëjnë të qartë ndihmën e përdorur.

Lidhja me aplikacionin mobile

- Demonstr problem, arkitekturën, Auth, CRUD dhe modulin unik.
- Shpjego një vendim teknik, një gabim dhe si e verifikove zgjidhjen.
- Deklaro kufizimet dhe kontrollo versionin e publikuar në telefon.

Shembull praktik

Një koleg ndjek README-n dhe e nis aplikacionin me konfigurimin e vet, pa sekrete të autorit në repository.

Pyetje ushtruese

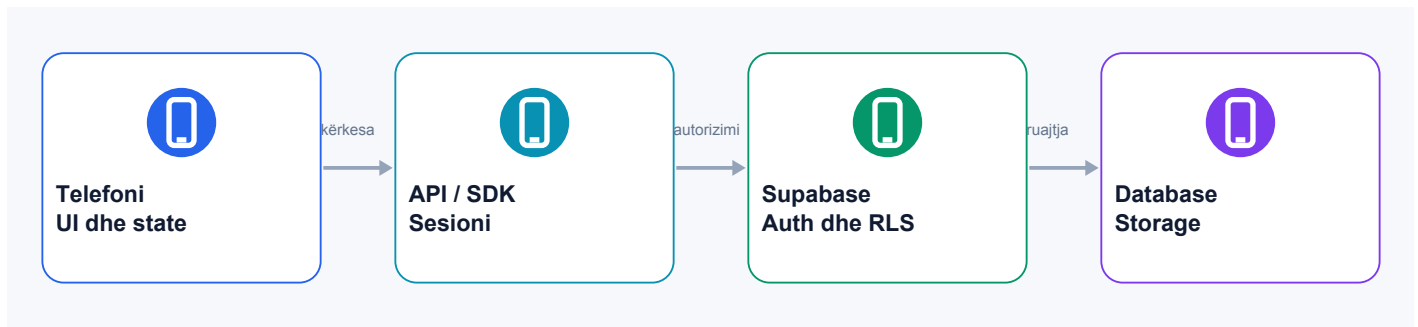
Çfarë duhet të përmbajë demonstrimi final?

Përgjigjja e modelit

Problemin që zgjidh aplikacioni, stack-un, arkitekturën dhe një rrjedhë funksionale me Auth, CRUD dhe modulin unik. Shpjego një vendim teknik, një gabim të zgjidhur, provat dhe përdorimin e AI-së. README-ja lejon riprodhimin e projektit dhe kufizimet shpjegohen qartë.

Nga telefoni te projekti i plotë

Një rrjedhë që bashkon ndërfaqen, identitetin, të dhënat dhe provat



Modeli mendor

Klienti merr input dhe shfaq gjendjen. Auth identifikon përdoruesin, API-ja bart kërkesën dhe lejet me RLS mbrojnë të dhënat. Edge Functions kryejnë logjikë serveri kur nevojitet; Realtime dhe push plotësojnë komunikimin sipas rastit.

Ushtrim praktik për vetëkontroll

Përdorni modulën tuaj të shënimeve me fusha `id`, `owner_id`, `title`, `body` dhe `created_at`. Demonstroni në telefon.

- A krijon një shënim dhe lexon listën e vet.
- B nuk mund ta lexojë, ndryshojë ose fshijë shënimin e A-së, edhe përmes API-së.
- Titulli bosh refuzohet me mesazh të qartë.
- Rrjeti i ndërprerë nuk prodhon mesazh të rremë suksesi ose shënime të dyfishta.
- Pas rihapjes, të dhënat e ruajtura mbeten dhe aplikacioni tregon gjendjen e saktë.

Përgatitja e demonstrimit

Shpjegoni problemin, stack-un, arkitekturën, Auth, CRUD dhe modulën unik. Paraqitni një vendim teknik, një gabim të zgjidhur dhe provën që e verifikoi. README-ja duhet të lejojë nisjen e projektit; AI-log duhet të bëjë të qartë ndihmën e përdorur.

Ky ushtrim është për përgatitje dhe nuk shton një detyrë të re me pikë.

Tabela e krahasimeve kryesore

Dallime që duhet t'i shpjegoni me shembuj nga projekti

Koncepti	Kuptimi	Shembulli ose arsyeja
Authentication / Authorization	Identiteti / të drejtat	Login-i identifikon; lejet vendosin çfarë mund të bësh.
UI / State	Pamja / gjendja	Ekrani pasqyron input-in, të dhënat dhe statusin e kërkesës.
Filtër në UI / RLS	Paraqitje / kontroll në databazë	RLS mbron rreshtat edhe ndaj kërkesave jashtë UI-së.
Database / Storage	Rreshta / skedarë	Të dhënat dhe metadata / fotografi e dokumente.
Realtime / Push	Lidhje aktive / njoftim platforme	Përditësim gjatë përdorimit / njoftim jashtë aplikacionit.
Klient / Edge Function	Telefoni / serveri	UI dhe reagimi / logjika me sekrete dhe kontrolle.
Loading / Empty / Error	Në proces / bosh / dështim	Mungesa e rezultateve nuk është e njëjtë me gabimin.
Cache / Të dhëna qendrore	Kopje lokale / versioni në backend	Kopja lokale mund të jetë e vjetruar; sinkronizimi kërkon rregulla.
Sugjerim AI / Verifikim	Propozim / provë	Studenti shqyrton kodin dhe teston rezultatin.
README / AI-log	Riprodhim / transparencë	Nisja e projektit / ndihma e përdorur dhe provat.

Kontroll i shpejtë para provimit

- Mund ta vizatoj rrjedhën nga UI-ja te backend-i dhe të dhënat.
- Mund ta provoj ndarjen e të dhënave me dy përdorues.
- Mund të shpjegoj gabimet, riprovimin dhe kufizimet e projektit.
- Mund të demonstroj Auth, CRUD dhe modulin unik pa u mbështetur vetëm te AI-ja.

Burimet dhe vlerësimi

Syllabusi 2025/2026 dhe dokumentacioni për përsëritje

Autoriteti i lëndës

Ky material mbështetet në syllabusin origjinal Programimi për pajisje mobile 2025/2026. Syllabusi parashikon 10% pjesëmarrje dhe angazhim dhe 90% projekt final me Auth, CRUD, një modul unik, README dhe AI-log. Për studentët që nuk marrin pjesë, politika parashikon provim final me 100% në afatin e radhës.

Kushtet e afatit

Data, kohëzgjatja, forma e provimit dhe mjetet e lejuara komunikohen nga profesori. Pyetjet në këtë dokument janë ushtruese dhe nuk përbëjnë listë të garantuar të pyetjeve të provimit.

01 Syllabusi origjinal 2025/2026

<https://arbenl.github.io/lendet/2025-2026/mobile/materiale/syllabusi-mobile-2025-2026.pdf>

02 Materialet dhe udhëzimi Mobile

<https://arbenl.github.io/lendet/2025-2026/mobile/>

03 Supabase Auth

<https://supabase.com/docs/guides/auth>

04 Supabase Row Level Security

<https://supabase.com/docs/guides/database/postgres/row-level-security>

05 Supabase API keys

<https://supabase.com/docs/guides/getting-started/api-keys>

06 Supabase Storage

<https://supabase.com/docs/guides/storage>

07 Supabase Realtime

<https://supabase.com/docs/guides/realtime>

08 Supabase Edge Functions

<https://supabase.com/docs/guides/functions>

09 Flutter

<https://docs.flutter.dev/>

10 React Native

<https://reactnative.dev/docs/getting-started>

11 Android Compose

<https://developer.android.com/compose>

Shënim pedagogjik

Shembujt përdorin Supabase dhe aplikacion shënimesh. Zbatoni të njëjtat parime në stack-un e projektit tuaj dhe shpjegoni zgjedhjet me prova nga aplikacioni.